SCIENCE AND DEVELOPMENT JOURNAL (SDJ) — EDITORIAL PORTAL · SUBMISSION DOCUMENTATION

06 · Testing Report

SDJ Editorial Portal — Testing Report

Project	SDJ Editorial Portal — an editorial portal for the Science and Development Journal (SDJ), published by the College of Basic and Applied Sciences, University of Ghana
Document	06 — Testing report
Author	Roger Koranteng Obeng, student ID 22424140
Assessor	Prof. Solomon Mensah
Date	2026-08-12, revised 2026-08-14 against the running system
Status	Authoritative. Every figure in this document was produced by running the commands it cites — `make check`, `make integration`, `npx vitest run`, and a live session against the deployed frontend — and is reproducible by running the same commands against the same commit; the suite counts are from the CI run named in section 3. Where a figure could not be produced this way, that is stated rather than estimated.

System under test Frontend: `https://ugjcs-frontend.vercel.app` · API: `https://tsxsbf9rzp.us-east-1.awsapprunner.com`

1. Testing strategy

The portal is built as a hexagonal architecture: a framework-free domain at the centre, an application layer orchestrating use cases against ports, and infrastructure adapters (PostgreSQL, S3, JWT, Argon2) at the edge, fronted by a FastAPI HTTP boundary and a Next.js Backend-For-Frontend. The test strategy is layered to match, because each layer fails in a different way and needs a different kind of test to catch it.

Layer	What it is	Why it exists
Unit — domain (`tests/unit/domain`)	Pure tests against `ugjcs.domain`, with no database, no HTTP, no mocks of infrastructure — the domain imports none of it, and `.importlinter`'s `domain-purity` contract enforces that mechanically.	The domain carries the manuscript lifecycle, the hash chain, blinding and RBAC policy — the parts of the system where correctness matters most and where a test can run in microseconds with no fixture cost.
Property-based (`tests/unit/domain/test_invariants.py`)	Hypothesis-generated inputs asserting invariants that must hold for *every* input, not the handful an author thinks to write by hand: no transition leaves a terminal state, publication is reachable only through acceptance and scheduling, a chain built by `append` always verifies, removing any non-terminal event always breaks verification, blinding never leaks an author id for any title/abstract/keyword combination.	Example-based tests only prove the examples chosen; property tests prove the property, over a search space the author did not have to enumerate.
Unit — application, API, db, security (`tests/unit/{application,api,db,security}`)	Application services against fake ports; FastAPI routers against a `FakeUnitOfWork` and dependency overrides; SQLAlchemy row↔aggregate mappers in isolation; password hashing and JWT issuance.	Each adapter's own logic — request validation, authorization wiring, row mapping, token signing — has failure modes independent of the domain and independent of a live database.

Layer	What it is	Why it exists
Integration (`tests/integration`, `-m integration`)	The same repositories, unit of work and triggers running against a **real PostgreSQL in a `testcontainers` container** — not a mock, not SQLite.	Several defects in this project (below) exist only at the boundary between the domain's in-memory model and what PostgreSQL actually does — `timestamptz` normalisation, trigger firing semantics, foreign-key `RESTRICT` behaviour. A mocked repository cannot fail in these ways, so it cannot catch them.
Contract tests (`.importlinter`, `mypy --strict`)	Two import-linter contracts (`domain-purity`, `layers`) plus strict static typing, run as part of `make check`.	These are architectural tests: they fail the build if a future change routes a dependency the wrong way, independent of any behaviour a functional test would check.
Route audit (`tests/unit/api/test_route_audit.py`)	Walks the *live* FastAPI route table built by `create_app()` and asserts every non-public route carries an authorization dependency somewhere in its dependency tree.	Its own docstring states the reason directly: a hand-maintained checklist of "which routes need auth" is exactly the kind of artefact a future change forgets to update. Walking the actual route table means a new unprotected route fails this test the moment it is added, not the moment someone remembers to check a list.
End-to-end	Exercised manually against the deployed system with a Playwright-driven browser session (section 5), not as a committed automated suite.	`frontend/package.json` declares `@playwright/test` and `@axe-core/playwright` as dependencies and a `test:e2e` script, but no `playwright.config.ts` or spec files exist in the repository at the time of writing — this is stated plainly in section 6 rather than implied by the tooling being present.
Manual acceptance testing	Scripted, role-scoped scenarios run against the live deployment with named judge accounts (section 5).	Automated gates check what someone thought to assert. Three domain lifecycle methods were fully implemented and unit-tested but reachable by no API route (section 4) — a defect only visible by using the system as an actual actor would.

2. Test environment and tooling

Concern	Tool	Configuration
Backend test runner	pytest 9.1.1 (`pytest-asyncio`, `pytest-cov`)	`backend/pyproject.toml` — `asyncio_mode = "auto"`; integration tests marked `integration` and deselected by default (`addopts = "-q --strict-markers -m 'not integration'"`).
Property-based testing	Hypothesis 6.165.3	`tests/unit/domain/test_invariants.py`; chain-invariant properties run at `max_examples=100` (or 50 for the deletion property), state-machine properties over the full enum space.
Real-database integration	testcontainers 4.15.0 (`testcontainers[postgres]`)	`tests/integration/conftest.py` spins up a genuine PostgreSQL container per test session; CI instead points at a `postgres:16` service container (`.github/workflows/backend-ci.yml`) so the same tests run without Docker-in-Docker.
Linting	ruff 0.16.2	`[tool.ruff.lint] select = ["E","F","I","N","UP","B","A","C4","SIM","RUF"]`, line length 100, target `py313`.
Static typing	mypy 2.3.0, strict mode	`[tool.mypy] strict = true`, `files = ["src","tests"]` — tests are type-checked, not only source.

Concern	Tool	Configuration
Architecture contracts	import-linter ≥2.13	`.importlinter` — `domain-purity` (forbids `fastapi`, `sqlalchemy`, `pydantic`, `boto3`, `os`, `io`, `socket`, `logging`, `asyncio`, and thirteen other framework/I/O modules from `ugjcs.domain`) and `layers` (`api → infrastructure → application → domain`, dependencies point inward only).
Coverage	coverage.py via `pytest-cov`, branch mode on (`[tool.coverage.run] branch = true`)	Gate at `--cov-fail-under=85`, scoped to `src/ugjcs/domain` and `src/ugjcs/application` only.
Frontend unit/component tests	Vitest 4.1.10, Testing Library, jsdom	`frontend/vitest.config.ts`; 20 test files under `src/**/*.test.{ts,tsx}` covering session-cookie sealing, Route Handler validation, and component rendering (blinding, status badges, forms, keyboard operability).
Frontend e2e (declared, not committed)	`@playwright/test` 1.62.1, `@axe-core/playwright` 4.13.0	Present in `package.json` as `npm run test:e2e`; no config or spec files exist yet (section 6).
CI	GitHub Actions, `.github/workflows/backend-ci.yml`	Two jobs, `check` and `integration`, both gating on `push` / `pull_request` to `main` / `master`.

Why the coverage gate is scoped to domain and application only

`backend/Makefile`'s `check` target carries this comment verbatim:

> The coverage gate covers domain and application only. Infrastructure adapters are verified by the integration suite, which needs Docker and is deselected here; measuring them in a run that excludes their tests would only reward mock-based tests that assert nothing. `make integration` reports adapter coverage separately.

In other words: measuring infrastructure coverage against a run that has already excluded the infrastructure tests would report a number with no information content — it would credit whatever incidental coverage the domain/application tests happen to produce by importing infrastructure modules, not coverage from tests that actually exercise them. `make integration` reports the infrastructure figure honestly, against the suite built to exercise it, with the gate set to `--cov-fail-under=0` because that run's purpose is behavioural verification against a real database, not a coverage target.

CI pipeline gates

`.github/workflows/backend-ci.yml` runs two jobs on every push and pull request against `main` / `master`:

- **`check`** — `uv sync --locked`, then `make check`: ruff lint, ruff format check, mypy strict, import-linter, and the unit suite with the 85% coverage gate.
- **`integration`** — brings up a `postgres:16` service container, applies the Alembic migration **up, then down, then up again** (verifying reversibility, not just forward application), then runs `make integration` against it.

A pull request cannot merge past either job failing.

---

3. Test cases

The counts below are what the suites reported on 2026-08-14, against the current commit, run from `backend/` and `frontend/`:

```
$ make check       → 402 passed, 84 deselected. Coverage: 90.03% (gate: 85%)
$ make integration → 84 passed, 402 deselected
$ npx vitest run   → 79 passed (20 files)
```

402 + 84 = 486 backend tests across the two runs. `--strict-markers` and the `integration` marker partition the suite so each test runs in exactly one of the two, which is why each run reports the other's count as deselected. The frontend adds 79 more, for 565 in total.

Both backend figures are from CI run 31791711326 on 2026-08-14, which reports `collected 486 items / 402 deselected / 84 selected` then `84 passed` for the integration job, and `402 passed` at 90.03% coverage for the gates job.

On the integration figure. These 84 tests stand up a real PostgreSQL 16 in a throwaway container via `testcontainers`, so they cannot run on a machine without a Docker daemon. On such a machine every one of them errors at setup rather than failing, which is a different and more honest signal than a false pass, and it is why this report cites the CI run rather than a local one. CI provides Postgres as a service container and additionally verifies that the Alembic migration chain applies, reverses to base, and re-applies.

The tables below draw specific, representative cases from the suites rather than reproducing all 486. The full set is in `backend/tests/`, one file per module under test.

3.1 Functional testing — domain lifecycle and rules

Test case	Expected result	Actual result	Pass/Fail
New manuscript starts in `DRAFT` (`test_new_manuscript_starts_in_draft`)	Status is `DRAFT`	`DRAFT`	Pass
Submitting twice (`test_cannot_submit_twice`)	Second submission raises an illegal-transition error	Raises	Pass
Desk rejection once under review (`test_desk_rejection_is_illegal_once_under_review`)	Rejected as illegal	Rejected	Pass
A decision below the minimum review count (`test_a_decision_requires_the_minimum_review_count`)	Raises for each decision type below quorum	Raises	Pass
Reaching the review quorum (`test_review_quorum_closes_the_review_round`)	Round closes automatically	Closes	Pass
Resubmission by a non-corresponding author (`test_only_the_corresponding_author_may_resubmit`)	Refused	Refused	Pass
Resubmission increments version, resets review count (`test_resubmission_increments_the_version_and_resets_review_count`)	Version +1, count reset to 0	Confirmed	Pass
Publication attempted before scheduling (`test_publication_requires_an_issue`)	Raises	Raises	Pass
Accepted → scheduled → published, in order (`test_accepted_manuscript_can_be_scheduled_then_published`)	Succeeds end to end	Succeeds	Pass
Withdrawal by a non-corresponding co-author (`test_a_co_author_who_is_not_corresponding_cannot_withdraw`, `test_policies.py`)	Refused	Refused	Pass
Every `ManuscriptStatus` appears in the transition table (`test_every_status_appears_in_the_table`)	No status is undeclared	Confirmed	Pass

Test case	Expected result	Actual result	Pass/Fail
Legal transitions permitted, shortcuts forbidden, over the full status × status matrix (`test_lifecycle_permits_expected_transitions`, `test_lifecycle_forbids_shortcut_transitions`)	Table matches exactly	Matches	Pass
Tracking code formatting and round-trip parsing (`test_tracking_code_formats_year_and_zero_padded_sequence`, `test_tracking_code_parses_its_own_output`)	`SDJ-YYYY-NNNN`, and `parse(format(x)) == x`	Confirmed	Pass
Role policy: reviewer may not decide, author may not screen, editor may not publish (`test_reviewer_may_not_decide`, `test_author_may_not_screen`, `test_editor_may_not_publish`)	All denied	All denied	Pass
Multiple roles grant the union of permissions (`test_multiple_roles_grant_the_union_of_permissions`)	Union, not intersection	Confirmed	Pass
PDF magic-number validation, independent of client-supplied content type (`test_a_client_supplied_content_type_cannot_substitute_for_the_magic_number`)	Rejected on bytes, not on header	Rejected	Pass
Storage keys carry no title or author-identifying text (`test_keys_carry_no_title_or_author_identifying_text`)	Confirmed	Confirmed	Pass

3.2 Property-based testing (Hypothesis, `test_invariants.py`)

Test case	Expected result	Actual result	Pass/Fail
No transition ever leaves a terminal state, over all 121 (source × target) status pairs	Holds for every pair	Holds	Pass
`PUBLISHED` reachable only from `SCHEDULED`, `SCHEDULED` only from `ACCEPTED` — chained, so `PUBLISHED` is unreachable without acceptance	Holds	Holds	Pass
A chain built entirely through `append` always verifies, over 100 generated payload sequences (size 1–12)	`verify()` raises nothing	Raises nothing	Pass
Removing any event except the last always breaks verification, over 50 generated (chain, victim-index) pairs	`ChainBrokenError` raised every time	Raised every time	Pass
Truncating the chain's tail is **not** detected (pinned known limitation, TD-04)	`verify()` accepts the truncated chain	Accepts	Pass — by design; see section 6
The blinded projection never carries the author id, for any generated title/abstract/keyword combination	Author id absent from the serialised projection	Absent	Pass
`canonical_bytes()` always parses back as valid JSON, over 100 generated payloads	Round-trips	Round-trips	Pass

3.3 Integration testing (real PostgreSQL via testcontainers)

Test case	Expected result	Actual result	Pass/Fail
A stored manuscript reads back with its roles and authors intact (`test_a_stored_manuscript_can_be_read_back`, `test_row_preserves_author_order`)	Exact round trip	Confirmed	Pass

Test case	Expected result	Actual result	Pass/Fail
`UPDATE` on `editorial_events` (`test_updating_an_event_is_rejected_by_the_database`)	Rejected by a trigger, error matches `"append-only"`	Rejected	Pass
`DELETE` on `editorial_events` (`test_deleting_an_event_is_rejected_by_the_database`)	Rejected	Rejected	Pass
`TRUNCATE TABLE editorial_events` (`test_truncating_the_event_log_is_rejected`)	Rejected by a statement-level trigger	Rejected	Pass — closes TD-13, see section 4
Deleting a manuscript that has audit events (`test_deleting_a_manuscript_with_events_is_refused`)	Refused by `ON DELETE RESTRICT`	Refused	Pass
A persisted hash chain verifies after a round trip through Postgres (`test_a_persisted_chain_verifies`)	Verifies	Verifies	Pass
The chain stays consecutive across separate transactions (`test_the_chain_stays_consecutive_across_separate_transactions`)	Confirmed	Confirmed	Pass
Replaying a rotated refresh token revokes the entire token family (`test_replaying_a_rotated_refresh_token_revokes_the_entire_family`)	Whole family revoked	Revoked	Pass
A role revoked after a token was issued takes effect immediately (`test_roles_revoked_after_a_token_was_issued_take_effect_immediately`)	New request re-checks roles from the database, not the token	Confirmed	Pass
An expired token is refused (`test_an_expired_token_is_refused`)	Refused	Refused	Pass
A successful login upgrades an outdated password hash (`test_a_successful_login_upgrades_an_outdated_password_hash`)	Rehashed to current parameters	Confirmed	Pass
Migration applies, reverses, and reapplies cleanly (CI step, `alembic upgrade head` → `downgrade base` → `upgrade head`)	No error at any step	No error	Pass
Expected tables and append-only triggers exist post-migration (`test_the_expected_tables_exist`, `test_the_append_only_triggers_are_installed`)	Present	Present	Pass
An assignment is visible to both the reviewer and the editor who made it (`test_an_assignment_is_visible_to_both_parties`)	Visible to both	Confirmed	Pass
Assigning the same reviewer twice is rejected (`test_assigning_the_same_reviewer_twice_is_rejected`)	Rejected	Rejected	Pass

3.4 System / API testing (FastAPI TestClient, fake unit-of-work)

Test case	Expected result	Actual result	Pass/Fail
A non-PDF upload (`test_a_non_pdf_upload_is_rejected_with_415`)	`415 Unsupported Media Type`	`415`	Pass
An oversized upload (`test_an_oversized_upload_is_rejected_with_413`)	`413 Payload Too Large`	`413`	Pass

Test case	Expected result	Actual result	Pass/Fail
A reviewer without the author role attempts to submit (`test_a_reviewer_without_the_author_role_cannot_submit`)	`403`	`403`	Pass
Retrieving someone else's manuscript (`test_retrieving_someone_elses_manuscript_is_forbidden`)	`403`	`403`	Pass
A malformed tracking code (`test_a_malformed_tracking_code_is_404_not_a_500`)	`404`, not an unhandled `500`	`404`	Pass
A reviewer attempts the editorial screening queue (`test_a_reviewer_may_not_see_the_screening_queue`)	`403`	`403`	Pass
A plain editor attempts to schedule or publish (`test_a_plain_editor_cannot_schedule`, `test_a_plain_editor_cannot_publish`)	`403` — editor-in-chief only	`403`	Pass
Publishing before scheduling (`test_publishing_without_scheduling_first_is_a_conflict`)	`409 Conflict`	`409`	Pass
Scheduling the same volume/number twice (`test_scheduling_the_same_volume_and_number_twice_yields_the_same_issue_id`)	Idempotent — same `IssueId` both times	Confirmed	Pass
An illegal domain transition surfaces as `409` RFC 9457 Problem Details (`test_illegal_transition_becomes_409_problem_json`)	`409`, `application/problem+json` body	Confirmed	Pass
An authorization denial surfaces as `403` Problem Details (`test_authorization_denied_becomes_403`)	`403`	`403`	Pass
CORS: an allowed origin gets CORS headers, a disallowed one gets none (`test_an_allowed_origin_receives_cors_headers`, `test_an_origin_not_on_the_allowlist_receives_no_cors_header`)	Confirmed both ways	Confirmed	Pass
`GET /health` reports OK without touching the database (`test_health_reports_ok_without_touching_the_database`)	`200`, no DB dependency in the route	Confirmed	Pass
An unassigned reviewer fetches a manuscript's document (`test_an_unassigned_reviewer_cannot_fetch_the_document`)	`403`	`403`	Pass
Route audit: every non-public route in the live route table carries an authorization dependency (`test_every_non_public_route_carries_an_authorization_dependency`)	Empty unprotected-route list	Empty	Pass
Route audit: the public allowlist is not accidentally matching everything (`test_the_public_allowlist_is_not_accidentally_empty`)	≥8 protected routes found	Confirmed	Pass
Route audit: `/api/v1/archive*` is genuinely public, not merely allowlisted (`test_the_archive_prefix_genuinely_has_no_authorization_dependency`)	No auth dependency present	Confirmed	Pass

3.5 Security testing

Test case	Expected result	Actual result	Pass/Fail
A stored password hash is never the plaintext (`test_a_hash_is_not_the_password`, `test_the_stored_password_hash_is_not_the_plaintext_password`)	Hash ≠ plaintext	Confirmed	Pass
Two hashes of the same password differ (salted) (`test_hashing_is_salted_so_two_hashes_differ`)	Differ	Differ	Pass
A weaker legacy hash is flagged for rehashing (`test_a_weaker_hash_needs_rehashing`)	Flagged	Flagged	Pass
A token signed with another secret is refused (`test_a_token_signed_with_another_secret_is_refused`)	Refused	Refused	Pass
A tampered token is refused (`test_a_tampered_token_is_refused`)	Refused	Refused	Pass
An access token cannot be used as a refresh token (`test_an_access_token_cannot_be_used_as_a_refresh_token`)	Refused	Refused	Pass
A refresh token is stored only as a hash, never in plaintext (`test_a_refresh_token_is_stored_only_as_a_hash`)	Confirmed	Confirmed	Pass
Reviewer-facing responses never serialise author identifiers, under distinctive sentinel UUIDs planted for the purpose (`test_my_assignments_never_serialises_author_identifiers`, `test_blinding_leak.py`)	No sentinel value present anywhere in the raw response body	Absent	Pass
The manuscript type returned to a reviewer exposes exactly the permitted field set, no more, no fewer (`test_the_manuscript_returned_by_my_assignments_is_the_blinded_type`)	`{tracking_code, title, abstract, keywords, version, status}` exactly	Exact match	Pass
A registration for an existing email raises and sends no second verification message (`test_registering_an_existing_email_raises_and_sends_no_second_message`)	No enumeration signal via message count	Confirmed	Pass
An unknown email at login fails identically to a wrong password (`test_unknown_email_is_rejected_identically_to_a_wrong_password`)	Same response either way	Confirmed	Pass
`canonical_bytes()` hashes the same instant identically regardless of UTC offset (`test_the_same_instant_hashes_identically_regardless_of_offset`)	Identical bytes	Identical	Pass — regression test for TD-12, see section 4
`UPDATE` / `DELETE` / `TRUNCATE` on the audit log, and a manuscript delete with attached events	All rejected at the database level	All rejected	Pass — see section 3.3, section 4

3.6 Frontend unit/component testing (Vitest)

Test case	Expected result	Actual result	Pass/Fail
Session cookie round-trips user identity and tokens through sealing (`session.test.ts`)	Round-trips	Round-trips	Pass
A session sealed with the wrong password never unseals to the original payload (`session.test.ts`)	Fails closed	Fails closed	Pass

Test case	Expected result	Actual result	Pass/Fail
`BlindedManuscriptView` never renders author identity, even if a payload is deliberately smuggled in upstream (`blinded-manuscript-view.test.tsx`)	No author field rendered	Confirmed	Pass
`DecisionForm` offers desk rejection only from `under_screening` , renders nothing once no legal decision exists (`decision-form.test.tsx`)	Confirmed both cases	Confirmed	Pass
Login page surfaces the RFC 9457 problem detail returned by the login Route Handler on failure (`login-page.test.tsx`)	Problem `title` / `detail` shown to the user	Confirmed	Pass
Login form is fully keyboard-operable — Tab order reaches email, password, submit (`login-page.test.tsx`)	Confirmed	Confirmed	Pass
`POST /api/manuscripts` Route Handler rejects an invalid body before ever calling upstream (`route.test.ts`)	Upstream not called	Not called	Pass
Clicking a demo-account chip fills the email and leaves the password empty and focused (`login-page.test.tsx`)	Email set, password empty and focused	Confirmed	Pass
A seeded address never carries across a switch to the sign-up form (`login-page.test.tsx`)	Email cleared	Cleared	Pass
The prototype notice precedes the form in document order (`login-page.test.tsx`)	Notice first	Confirmed	Pass

Frontend suite: **20 test files, 79 cases, all passing** under `npx vitest run` .

One environment note, since it caused a false alarm once. Two suites import `frontend/src/lib/env.ts` , which parses `process.env` at import time, so they fail with a Zod error unless `API_BASE_URL` and `SESSION_SECRET` are present. `vitest.config.ts` feeds them in through `loadEnv` , which reads `.env*` files; a checkout without a local `.env.local` therefore sees those suites error rather than fail. Copying `.env.local.example` resolves it.

3.7 Feature suites added after the original build

The two waves that followed the 48-hour build brought their own suites. These are the files, with the property each one exists to pin:

Suite	What it pins
`test_billing_router.py`	An invoice opens on an accept decision and only on an accept decision; a repeated accept never double-bills; mock and real gateway modes are distinguishable on the wire; only the corresponding author may settle, only the Editor-in-Chief may waive
`test_invoice_repository.py` (integration)	Invoice persistence and status transitions against real Postgres
`test_admin_router.py`	Every admin route is closed to non-administrators; the administrator role can be neither granted nor revoked; capacity is bounded 1 to 10; an administrator cannot deactivate themselves
`test_provenance_router.py`	An intact chain verifies and reports its head; a tampered interior event is reported as broken; payloads and actor ids are never exposed
`test_citation_router.py`	BibTeX and RIS are well formed; an unknown or missing format is 422; a DOI-shaped identifier is present
`test_editorial_analytics.py`	Aggregates over an empty desk return nulls rather than zero rates; reviewer performance reports workload and turnaround; deadlines carry a server-computed overdue flag; reviewers cannot read any of it
`test_archive_search_fulltext.py`	Body-text matches carry a snippet, title and keyword matches carry null; publishing extracts PDF text into the search column; an unreadable document never blocks a publish

Suite	What it pins
`test_submission_preflight.py`	Submission and resubmission report stripped metadata keys and flag author names still present in body text, without dropping any `ManuscriptOut` field
`test_certificate_router.py`, `test_certificate_auth.py`	A certificate is a PDF, reachable by editor and Editor-in-Chief, and names no reviewer and no confidential comment
`test_fulltext_search.py`, `test_due_at_migration.py` (integration)	The `tsvector` column and the `due_at` migration behave against real Postgres

Two of these deserve a note because they assert an absence rather than a presence. `test_event_payloads_and_actor_ids_are_never_exposed` and `test_the_certificate_never_names_a_reviewer_or_leaks_confidential_comments` both guard the double-blind guarantee at points where a new feature could plausibly have broken it: a public verification endpoint and a downloadable PDF are exactly the kind of additions that leak an identifier by accident. Asserting the absence directly is cheaper than hoping a reviewer notices.

---

4. Defects found and corrective action

This is the section with the most to learn from. Every entry below was found either by a human or an agent reading code against what it claimed to do, by mutation testing, or by exercising the deployed system — none were found by a coverage number or a passing test suite on its own. Full Debt → Cause → Impact → Priority → Resolution entries for the still-open items are in Technical_Debt_Plan.pdf; this section retells them from the testing perspective — what the test evidence looked like, and how the gap was closed.

4.1 Mutation testing found the hash chain's chaining step was untested

What was found. During the final review, deleting the single line in `chain_hash` that folds the predecessor's hash into the digest left **all tests passing**. Without that line, `event_hash` is a per-event checksum with no dependency on anything before it — altering or removing an earlier event would no longer invalidate anything after it, which defeats the entire purpose of a hash *chain*. The suite was at 100% line and branch coverage on `hashchain.py` at the time.

How it was found. Manual mutation testing: deliberately deleting a line believed to be load-bearing and re-running the suite, rather than trusting the coverage figure. This is recorded in Technical_Debt_Plan.pdf (TD-11) as one of four mutations that survived a 100%-covered suite.

Why it mattered. It is the sharpest illustration in this project of the gap between "every line executed" and "every behaviour asserted." Coverage measured that the line ran; it said nothing about whether removing it changed an observable outcome, because no test happened to construct a case where the predecessor's hash mattering was the only thing being checked.

What was done. `test_the_predecessor_hash_changes_the_digest` was added directly: `chain_hash(subject, GENESIS_HASH) != chain_hash(subject, "f" * 64)` — the same event, two different predecessor hashes, asserted to produce different digests. This test exists specifically so the deleted-line mutation is caught; it is written as a test *about* the mutation, not merely a test that happens to cover the line.

4.2 A UTC-offset representation would have produced a false tamper alert

What was found. `canonical_bytes()` originally hashed `occurred_at.isoformat()`, which includes whatever UTC offset the Python `datetime` carried. PostgreSQL's `timestamptz` normalises any offset to UTC on storage. An event recorded with a non-UTC offset (e.g. `+05:00`) would therefore hash one way in memory, before persistence, and differently after a round trip through the database — because the timestamp that comes back is the same instant in a different textual representation.

How it was found. Not by any test — every existing test used a UTC datetime throughout, so the suite was structurally blind to the defect. Four independent reviews of the in-memory code did not catch it either. It was found by reasoning about the storage boundary specifically: asking what happens to a value that crosses into PostgreSQL and back, not just what the Python code does in isolation.

Why it mattered. This is worse than an undetected tamper — it is a **false positive**. `verify()` would report tampering on an event nobody touched, purely because of a representation mismatch. A false alarm destroys confidence in every subsequent genuine detection; a security control that cries wolf gets ignored or disabled.

What was done. `canonical_bytes()` now normalises `occurred_at` to UTC before hashing, so the offset cannot reach the digest by construction. The regression test `test_the_same_instant_hashes_identically_regardless_of_offset` (`test_events.py`) pins this: the same instant, expressed as `2026-08-12T09:30:00+00:00` and as `2026-08-12T14:30:00+05:00`, must hash identically. Recorded as resolved TD-12.

4.3 `TRUNCATE` bypassed the append-only trigger entirely

What was found. A `BEFORE UPDATE OR DELETE ... FOR EACH ROW` trigger protected `editorial_events`, and was verified firing against a live database — `UPDATE` and `DELETE` were both confirmed rejected. But PostgreSQL never fires **row-level** triggers on `TRUNCATE`. A single `TRUNCATE TABLE editorial_events` would erase the entire audit log, with no error, no row-level check ever invoked.

How it was found. By asking a broader question of a control already believed correct: "is this claim true for every statement class that removes rows?", not "does the trigger work?" The trigger's own review had only checked the two statement types it was written to intercept.

Why it mattered. The system was claiming a tamper-evident audit trail while leaving the single most direct way to destroy it completely unguarded — one SQL statement, no privilege escalation required beyond whatever already had write access to the table.

What was done. A statement-level `BEFORE TRUNCATE ... FOR EACH STATEMENT` trigger was added, in both the Alembic migration and the test fixtures. `test_truncating_the_event_log_is_ rejected` (`tests/integration/test_append_only.py`) asserts `TRUNCATE` now raises a `DBAPIError` matching `"append-only"`, run against the real container, not a mock. Recorded as resolved TD-13.

4.4 coverage.py does not treat an inline ternary as a branch point

What was found. `tests/unit/db/test_mappers.py` reported no missing branches for the mappers module despite branch coverage being enabled (`branch = true`), yet `IssueId(row.issue_id) if row.issue_id is not None else None` had, in fact, only ever been exercised on one arm — the populated arm never ran in any test that existed before this was found.

How it was found. Reading the module's logic against what the coverage report claimed, rather than trusting "0 branches missing" as proof of exhaustive exercise. Recorded in Technical_Debt_Plan.pdf (TD-11) as the register's second illustration of coverage as a weak signal — a companion finding to the mutation-testing result in section 4.1, found the same review pass.

Why it mattered. "100% coverage, zero branches missed" is a claim readers reasonably take to mean both arms of every branch ran. It did not mean that here, because coverage.py's branch instrumentation does not model a Python conditional expression (`x if c else y`) as a two-arm branch the way it models an `if` / `else` statement. A reader trusting the coverage report alone would believe the populated-issue-id path was verified when it was not.

What was done. `test_round_trip_restores_a_populated_issue_id` (`test_mappers.py`) exists specifically to exercise the previously-untested arm, independent of what the coverage tool reports for it.

4.5 Three lifecycle methods were implemented and tested, but reachable by no route

What was found. `Manuscript.resubmit`, `Manuscript.schedule` and `Manuscript.publish` were fully implemented in the domain and covered by unit tests (section 3.1) — but at one point in the build, no API route in `ugjcs.api` called them. The domain logic was correct and verified in isolation; the system as a whole could not do the things those tests proved the domain capable of.

How it was found. Not by any automated test — every domain test for these methods passed. It was found by the project owner attempting to use the deployed system through the actual workflow (resubmit a revision, schedule an accepted paper, publish a scheduled issue) and discovering the corresponding action had no route to reach it.

Why it mattered. It is the clearest demonstration in this project that unit-level correctness and system-level completeness are different properties. A domain method can be implemented, unit-tested, type-checked, and covered at 100%, and still be dead code from the system's perspective if no adapter wires it in. No test in the suite — unit, integration, or route-audit — was positioned to catch a route's *absence*; `test_route_audit.py` proves every *existing* route is authorized, which is a different claim from every needed route existing.

What was done. The three routes were added to `ugjcs.api` (manuscript resubmission, editor-in-chief scheduling, editor-in-chief publication), each exercised by the system tests in section 3.4 (`test_the_corresponding_author_can_resubmit_with_a_revised_file`, `test_the_editor_in_chief_can_schedule_an_accepted_manuscript`, `test_the_editor_in_chief_can_publish_a_scheduled_manuscript`). The gap itself is the strongest argument in this report for section 5's manual acceptance pass being a required step, not an optional one: it is precisely the class of defect route-level and unit-level testing cannot see.

4.6 A container had an IAM route to S3 but no network route to reach it

What was found. In the deployed environment, the App Runner service held IAM permission to write to the document-storage S3 bucket, but ran in a network configuration with no route to reach the S3 endpoint. Document uploads hung rather than failing fast, until the platform's own health check judged the instance unresponsive and killed it.

How it was found. Only by exercising the deployed system's upload feature directly against the live API — no unit or integration test runs against the actual AWS network topology, so none could have surfaced this. `tests/unit/infrastructure/storage/test_s3_store.py` tests the `S3DocumentStore` adapter's logic (it requests a `PUT` and returns the generated presigned URL) against a mocked `boto3` client; that test is correct about what the adapter code does and had no way to observe that the network path to actually deliver that request did not exist in the deployed environment.

Why it mattered. It is a category of defect testing at any layer below "the deployed system, over the real network" cannot reach: infrastructure connectivity. IAM policy and network reachability are independent failure modes — a principal can be fully authorized to call an API and still have no path to the endpoint that serves it — and nothing short of actually attempting the call from the actual deployed environment exercises both at once.

What was done. The networking configuration was corrected so the App Runner service's egress path reaches the S3 endpoint. This is recorded here rather than as a technical debt register entry because it was a deployment configuration defect, fully resolved, not an accepted or scheduled trade-off; the lesson it leaves is procedural, not a residual risk: a feature is not verified until it has been exercised against the actual deployed infrastructure, not only against a mock of the SDK that infrastructure is reached through.

Summary

#	Defect	Found by	Class
4.1	Hash-chain step deletable with all tests passing	Mutation testing	Coverage/behaviour gap
4.2	UTC-offset timestamp would false-positive on tamper	Human reasoning about the storage boundary	Boundary-crossing representation defect
4.3	`TRUNCATE` bypassed the append-only trigger	Human asking a broader question of a verified control	Incomplete threat coverage
4.4	Ternary's second arm never executed, coverage silent	Human reading logic against the coverage report	Tooling blind spot
4.5	Three domain methods reachable by no route	Owner using the deployed system	System-completeness gap invisible to unit tests
4.6	IAM permitted, network unreachable	Owner exercising the deployed feature	Infrastructure-layer defect invisible to mocked tests

5. User acceptance testing

UAT was run as scripted, role-scoped scenarios against the **live deployment** (`https://ugjcs-frontend.vercel.app`, backed by the API at `https://tsxsbf9rzp.us-east-1.awsapprunner.com`), using the judge accounts:

Role	Account	Password
Author	`author@sdj.test`	`Sdj-Author-2026!`
Reviewer	`reviewer@sdj.test`	`Sdj-Reviewer-2026!`
Editor	`editor@sdj.test`	`Sdj-Editor-2026!`
Editor-in-Chief	`eic@sdj.test`	`Sdj-EditorChief-2026!`
Administrator	`admin@sdj.test`	`Sdj-Admin-2026!`

Scenarios actually exercised for this report (live, browser-driven)

Scenario	Steps	Expected	Observed	Result
Author login and dashboard	Navigate to `/login`, submit `author@sdj.test` / `Sdj-Author-2026!`	Redirected to `/author`, listing this author's submissions with status	Redirected to `/author`; ten submissions listed with correct tracking codes and statuses (`Published`, `Revision requested`, `Submitted`, `Under review`)	Pass
Public archive reachable without authentication	Navigate to `/search` with no session	Search page renders, no login redirect	Page rendered directly, titled "Search · SDJ Editorial Portal"	Pass

Scenario	Steps	Expected	Observed	Result
Cross-role access denied at the routing layer	While authenticated as `author`, navigate directly to `/editor`	Access refused, not the editor queue	Redirected to `/`; no editor content rendered	Pass — matches `frontend/middleware.ts`: a session present without the required role redirects to `/`, distinct from the no-session case, which redirects to `/login` with the intended destination attached

The redirect on a denied role check is a deliberate design choice, not a UAT defect: `middleware.ts` distinguishes "no session" (→ `/login`, so the user can authenticate and retry) from "session present, wrong role" (→ `/`, which now forwards a signed-in user to their own workspace — somewhere useful rather than a bare error). The backend's own authorization check — exercised in section 3.4 and section 3.5 — is what is actually authoritative for every state-changing action regardless of what the frontend route guard does; the middleware is a routing-level convenience, not the security boundary.

Scripted scenarios per role (full acceptance script)

The remaining scenarios below constitute the acceptance script run manually across the roles during development and at each deployment; they are stated here as the acceptance criteria this system is expected to meet, cross-referenced to the automated test that pins the same behaviour where one exists, since re-running the full script for every role on every document revision is not repeated for this report beyond the live checks above.

Role	Scenario	Automated backstop
Author	Submit a new manuscript with a PDF; see it appear under "My submissions"	`test_an_author_can_submit_a_manuscript_with_a_pdf`
Author	Attempt to submit a non-PDF file; see a clear rejection	`test_a_non_pdf_upload_is_rejected_with_415`
Author	Resubmit a manuscript sent back for revision	`test_the_corresponding_author_can_resubmit_with_a_revised_file`
Author	Withdraw a submission before a decision is recorded	`test_withdrawal_is_permitted_before_a_decision`
Reviewer	See only manuscripts assigned to them, blinded	`test_my_assignments_lists_only_manuscripts_assigned_to_me`, `test_my_assignments_never_serialises_author_identifiers`
Reviewer	Submit a review with a recommendation and comments	`test_submitting_a_review_counts_it_and_records_the_content`
Reviewer	Attempt to open a manuscript with no assignment	`test_an_unassigned_reviewer_cannot_fetch_the_document`
Editor	Move a submitted manuscript into screening, then to review	`test_an_editor_can_begin_screening`, `test_a_decision_moves_the_manuscript_to_review`
Editor	Assign a reviewer to a manuscript	`test_assigning_a_reviewer_records_it`
Editor	Attempt to schedule or publish (denied — EiC only)	`test_a_plain_editor_cannot_schedule`, `test_a_plain_editor_cannot_publish`
Editor-in-Chief	Schedule an accepted manuscript into a volume/issue	`test_the_editor_in_chief_can_schedule_an_accepted_manuscript`

Role	Scenario	Automated backstop
Editor-in-Chief	Publish a scheduled manuscript	`test_the_editor_in_chief_can_publish_a_scheduled_manuscript`
Administrator	Open the accounts console at `/admin`; grant and revoke a role; set a reviewer's capacity; deactivate and reactivate an account	`test_granting_the_reviewer_role_adds_it`, `test_capacity_can_be_set_within_one_to_ten`, `test_an_administrator_can_deactivate_and_reactivate_another_account`
Reader (unauthenticated)	Browse the public archive, search, download a published paper	`test_the_archive_requires_no_authentication`, `test_search_finds_a_matching_paper`

6. What testing did not cover, stated plainly

- **No load or performance testing.** Nothing in this project measures throughput, latency percentiles, or behaviour under concurrent load. The design specification's NFR-08/NFR-09 performance objectives are unverified by any test in this suite — they were sized at design time, not confirmed against the running system.
- **No automated security scanning in CI.** `.github/workflows/backend-ci.yml` runs linting, type checking, an architecture contract, and the test suites — it does not run a dependency vulnerability scanner, a SAST tool, or a DAST pass against the deployed API. Security coverage in this project is what the tests in section 3.5 assert directly, and nothing beyond that.
- **No mutation testing in CI.** The mutation-testing finding in section 4.1 was a one-off manual review pass, not a repeatable gate. Technical_Debt_Plan.pdf (TD-11) records systematic mutation testing (`mutmut` or `cosmic-ray`) as future evolution, not as something this project currently runs.
- **No browser-matrix testing.** The live UAT pass in section 5 was run in one browser (a Chromium-based automated session). No cross-browser or cross-device matrix was exercised; `@axe-core/playwright` is present as a dependency for accessibility auditing but, as noted in section 1 and section 3.6, no Playwright spec files are committed to run it against.
- **No committed automated end-to-end suite.** `test:e2e` is declared in `package.json` but there is no `playwright.config.ts` or `tests/e2e/` directory in the repository. The acceptance evidence in section 5 substitutes a manual (and, for this report, live-browser-driven) pass for what an automated E2E suite would otherwise provide continuously; it is not equivalent to one, because it was not re-run on every change, only at the points recorded here.
- **The double-blind guarantee has a stated limit.** `blind()` strips `author_ids` and `corresponding_author_id` from the type a reviewer receives — proven for every input by the property test in section 3.2 and the sentinel-based leak tests in section 3.5 — but `title`, `abstract` and `keywords` are carried **verbatim**. An author's name in a title, or an abstract that reads "extending our earlier work in [Obeng 2025]," reaches the reviewer unchanged; nothing in this system detects or redacts self-identifying body text. This is recorded as TD-05 in the technical debt register: double-blind integrity depends partly on author compliance with submission guidance, not entirely on what the system enforces. Reliable automated redaction was judged a research problem in its own right, where a bad redaction that leaks a name would undermine the guarantee more than not attempting redaction at all.

7. Evaluation of the testing strategy

The honest conclusion, and the one the evidence in section 4 supports without qualification: **automated gates establish a floor and catch regressions; they did not find the defects that mattered most on this project.** Every serious defect recorded in section 4 was found by a human or an agent reading code against what it claimed to do, by mutation testing deliberately designed to distrust the coverage figure, or by using the running system as an actual actor would — never by a coverage number or a green test run on its own.

The clearest single data point is section 4.1: coverage on `hashchain.py` stood at 100% — every line, every branch — while the line that makes a hash chain a *chain*, rather than a list of independently checksummed events, could be deleted without failing a single one of 104 tests that existed at the time. A tamper-evidence guarantee that is the entire reason this subsystem exists was, for a period, unprotected by any test that would have noticed its removal. That is not a marginal case; it is the load-bearing property of the module, and coverage reported nothing wrong.

Three further findings sharpen the same point from different angles. Section 4.2 shows that 100% coverage inside a module says nothing about what happens at a boundary the module doesn't know it crosses — every test used a UTC datetime, so the suite was structurally blind to a defect that only a database round trip could expose, and the defect it hid was actively harmful (a false tamper alert), not merely a missed detection. Section 4.3 shows that a control verified against the cases its author thought to check — `UPDATE`, `DELETE` — said nothing about the case nobody asked about — `TRUNCATE` — despite the control having been "confirmed firing against a live database." section 4.4 shows the measurement tool itself has blind spots: an inline ternary's second arm never ran, and coverage.py's branch instrumentation does not model a conditional expression as a branch, so the report read as complete when it was not.

Section 4.5 and section 4.6 extend the same lesson past the test suite entirely. Section 4.5 — three fully implemented, fully unit-tested domain methods reachable by no API route — is a defect no unit test, integration test, or route-audit test was positioned to find, because each of those tests presupposes the thing under test is reachable; the question "does anything reach this code" sits above all of them, and was answered only by the owner using the deployed system as a user would. Section 4.6 — correct IAM policy, no network path to use it — sits a layer below every test in this suite: nothing here runs against the actual deployed network topology, so the gap was invisible until the feature was exercised against the real infrastructure and the upload hung.

None of this is an argument against the automated suite. 402 unit tests, 84 integration tests against a real database, 79 frontend tests, two architecture contracts, and a strict type checker catch an enormous amount of regression cheaply and continuously — exactly what section 1 claims for them, and exactly the floor Technical_Debt_Plan.pdf describes them as. But the register's own closing observation, produced independently of this report, states the same conclusion this section reaches from the test evidence directly: every inadvertent entry in the register was found by review of work that had already passed every automated gate — linting, strict typing, an architecture contract, and a full test suite at 100% coverage. Passing every gate available in this project is necessary and was, in every one of these cases, not sufficient. What found the defects that mattered was a human, or an agent acting as one, asking whether the code was still true to what it claimed — and, twice in this project, whether the *deployed system* still did what the code claimed once it left the test suite entirely.